Schema Linking via LoRA Fine-Tuning of Qwen2.5-1.5B-Instruct

CSE/DSC 234: Data-Centric AI and AI Engineering — Project 2

Sukruth Rao PID: A69042450
University of California San Diego

June 2026

Abstract

Schema linking identifies which tables and columns in a relational database are relevant to a natural language question, serving as a necessary precursor to text-to-SQL generation. We fine-tune `Qwen/Qwen2.5-1.5B-Instruct` using Low-Rank Adaptation (LoRA) on the SNAILS schema-linking benchmark (301 training examples, 101 validation examples, 9 databases). We design and execute a systematic ablation study via RapidFire AI's `RFGridSearch` API across nine configurations spanning prompt format, LoRA rank, learning rate, and target projection modules. We additionally identify and correct three inference pipeline bugs that silently suppressed score regardless of model quality.

1 Training Data Methodology

1.1 Source and Composition

The training set is a 301-example subset of the SNAILS benchmark, released directly with the project. Each example is a JSON object with fields `question_id`, `db_id`, `question`, `gold_sql`, and `schema_links`. Schema links were derived from the gold SQL using the provided `sql_to_schema_links.py` parser, which walks the SQL AST to extract all referenced table and column identifiers.

The gold labels are stored as a JSON object mapping table names to lists of column names:

```
{"CRASH": ["CRASHYEAR", "CASEID"], "VEHICLE": ["VIN", "MAKE"]}
```

Tables that are referenced but require no specific column (e.g., a bare `COUNT(*) FROM t`) are included with an empty list.

1.2 Schema Serialization

Each database schema is loaded from a Spider-format JSON file (`schemas/<db_id>.json`) containing `table_names_original`, `column_names_original`, `primary_keys`, and `foreign_keys`. At training time, the schema is converted to a Python dictionary mapping table names to column-name lists and then serialized using Python's default string representation inside the prompt (matching the exact format seen at inference).

1.3 Data Augmentation

The raw training set contains 301 examples. Because each configuration in our ablation study uses a different schema serialization format (Section **??**), we explored whether pre-generating all three format variants per example and training on the union would expose the model to a richer input distribution.

We implemented `augment_data.py`, which produces an `augmented_train.json` of 903 examples (301×3) by applying the *basic*, *PK/FK-annotated*, and *sorted* prompt templates to every training example. However, in the final experimental setup we train each configuration on a *single* format (matching the format used at inference for that configuration) rather than the combined pool. The rationale is that mixing formats in one training run conflates two distinct skills — format-invariant schema reasoning and format-specific token patterns — making it harder to attribute performance differences to any single variable. The augmented dataset is retained as a resource for future experiments.

1.4 Diagnostic Metrics

We track two signals during development to catch data and pipeline errors before committing training time:

1. **Empty-prediction rate.** After inference we count questions for which `schema_links` is an empty dict {}. A high rate (we observed 19/101 before pipeline fixes) indicates a parsing or model failure rather than genuine uncertainty.
2. **Hallucination count.** `eval.py` reports the number of predicted table and column identifiers that do not appear in the target database schema. Zero hallucinations (achieved consistently) confirms that our schema-normalization step correctly maps all model outputs to valid schema identifiers or discards them.

2 Final Pipeline Architecture

2.1 Base Model

We use `Qwen/Qwen2.5-1.5B-Instruct` [**?**]: a 1.5B-parameter instruction-tuned causal LM with a 32K-token context window. It is loaded in FP16 via `AutoModelForCausalLM.from_pretrained` with `device_map="auto"` and then wrapped with a PEFT LoRA adapter using `PeftModel.from_pretrained`.

2.2 LoRA Adapter Configuration

We apply Low-Rank Adaptation [**?**], which freezes W_0 and learns a rank-r decomposition $W = W_0 + \frac{\alpha}{r} BA$. The final submitted adapter uses the best configuration identified by the ablation study (Section **??**). Key fixed parameters are LoRA dropout = 0.1, bias = `none`, and FP16 training.

2.3 Prompt Template

All prompts follow the Qwen2.5-Instruct chat format (`<|im_start|>`/`<|im_end|>` delimiters), with a system prompt and a user turn. The system prompt specifies the schema-linking task and output type. The user turn contains three components: the serialized database schema, the natural-language question, and an instruction constraining the output format. The exact template used at inference matches the template used at training for the same configuration (Section **??**).

2.3.1 Format Modes

`main.py` accepts `--format {basic|pkfk|sorted}` so that the inference prompt always matches the training format for the chosen adapter.

basic. The schema is serialized as a plain Python dict. The instruction requires specific column names and provides a short example.

```
System: You are a schema-linking assistant. Given a question and a
        database schema, return ONLY a valid JSON object that maps
        table names to relevant column-name lists.

User:
Database schema: {'CRASH': ['CASEID', 'CRASHYEAR', ...], ...}

Question: What years does this data span?

Return a JSON object with only the relevant tables as keys and lists
of relevant column names as values. You MUST include specific column
names. Example: {"Orders": ["order_id", "total"], ...}
```

pkfk. Each column is annotated with its role in the schema: `(PK)` for primary keys and `(FK→Table)` for foreign keys. These structural cues may help the model identify join paths when a question spans multiple tables. The instruction reminds the model to *exclude* annotations from the output.

sorted. Tables and columns are sorted alphabetically before prompt construction. This ensures that tables with similar prefixes (e.g., `ACC_EM_*` and `ACC_HS_*` in NYSED_SRC2022) are adjacent in the prompt, which may reduce confusion.

2.4 Output Decoding

Generation uses greedy decoding (`do_sample=False`, `num_beams=1`) with `max_new_tokens=1024` for determinism and to avoid output truncation. The seed is fixed at 42 via `torch.manual_seed` and `random.seed`.

2.5 Post-Processing

2.5.1 JSON Parsing

The raw model response is parsed through a three-stage fallback:

1. **Direct parse.** `json.loads(response.strip())` handles clean outputs. If the result is a JSON list (e.g., the model wraps the answer in `[{''TableName'': ...}]`) it is converted to a dict via a key-lookup heuristic.
2. **Substring extraction.** The first balanced `{...}` block is extracted via bracket counting and parsed. This handles responses where the model emits reasoning text before the JSON object.
3. `json_repair` **fallback.** Applied to the full response; handles truncated output, single-quote dialects, and trailing commas.

2.5.2 Schema Normalization

Predicted table and column names are matched against the target schema *case-insensitively*. Hallucinated identifiers (no case-insensitive match in the schema) are discarded and tallied separately. If a predicted table matches but none of its columns do, the table is still emitted with an empty column list to preserve table-level credit under the evaluation metric.

2.5.3 Pipeline Bug Fixes

Three bugs in the original `main.py` silently suppressed scores:

1. **Output truncation.** `MAX_NEW_TOKENS` was 512. Although most gold schema-links are short, the model sometimes emits a brief preamble before the JSON. Increased to 1024.
2. **Type-unsafe JSON result.** The original `json.loads(repair_json(response))` did not check whether the result was a `dict`. If `repair_json` returned a list, the downstream `format_schema` call silently returned {} because it guards on `type(data) != dict`. Fixed by adding list-to-dict conversion.
3. **Dropped tables on non-list column values.** `get_simple_table_data` skipped any entry whose column value was not a Python list (e.g., the model emitting {`"CRASH": "CRASHYEAR"`}). This lost table-level credit. Fixed so that a string value becomes a single-element list and any other non-list type yields an empty column list (table still credited).

After these fixes the number of empty predictions fell from 19 to 2. The two remaining empty predictions are caused by the model hallucinating table names absent from the schema; this is a model quality issue.

3 Experimentation Methodology

3.1 Exploratory Phase

Before the structured ablation, we ran a series of exploratory experiments using RapidFire AI (`experkrj_16` through `experkrj_37`). The most informative finding was that very light LoRA configurations (e.g., $r = 4$, attention-only modules, $\sim$74 training steps, lr $= 2 \times 10^{-5}$) produce a validation leaderboard score of approximately 0.3983. Inspection of per-question metrics revealed:

- **Table score** was 0.5015 (acceptable), indicating the model identifies the right table for roughly half the questions.
- **Column score** was 0.2950 (poor), driven by low column precision: the model often outputs all columns of a table rather than only the relevant ones.
- Large databases with cryptic abbreviations (NTSB, 40 tables) caused consistent failures; the model defaulted to the most common table (`CRASH`) regardless of the question.

These findings motivated moving to a heavier, systematic ablation.

3.2 Ablation Study Design

We use RapidFire AI's `RFGridSearch` API to train nine configurations in a single experiment run, treating **A1** as the baseline and varying one axis at a time. The `RFGridSearch` wrapper manages model instantiation, data loading, and checkpoint saving, allowing us to launch all configurations with a single `experiment.run_fit()` call.

Table **??** lists all configurations.

Table 1: Ablation configurations. Each row changes one variable (bold) relative to baseline A1. *all* modules: {q,k,v,o,gate,up,down}_proj; *attn*: {q,k,v,o}_proj.

ID	Name	Axis tested	Format	Modules	r	α	Steps	LR
A1	A1_basic_r8	baseline	basic	all	8	16	240	10^{-4}
A2	A2_pkfk_r8	prompt format	**pkfk**	all	8	16	240	10^{-4}
A3	A3_sorted_r8	prompt format	**sorted**	all	8	16	240	10^{-4}
B1	B1_basic_r16	LoRA rank	basic	all	**16**	**32**	240	10^{-4}
B2	B2_basic_r32	LoRA rank	basic	all	**32**	**64**	240	10^{-4}
C1	C1_basic_lr2e4	learning rate	basic	all	8	16	240	$\mathbf{2 \times 10^{-4}}$
C2	C2_basic_lr5e5	learning rate	basic	all	8	16	240	$\mathbf{5 \times 10^{-5}}$
D1	D1_basic_attn	target modules	basic	**attn**	**16**	**32**	240	10^{-4}
E1	E1_pkfk_r16_150	best combination	**pkfk**	all	**16**	**32**	**150**	10^{-4}

3.3 Rationale for Each Axis

Prompt format (A1 vs. A2 vs. A3). The baseline (A1) presents the schema as a plain dictionary. PK/FK annotations (A2) add explicit structural cues about join relationships, which may help the model select columns that serve as join keys. Alphabetical sorting (A3) groups similarly-named tables adjacently (e.g., all `ACC_EM_*` and `ACC_HS_*` tables in NYSED_SRC2022), which may help the model distinguish between them.

LoRA rank (A1 vs. B1 vs. B2). Rank controls the number of trainable parameters ($(d+k)\times r$ per weight matrix). Higher rank gives the adapter more capacity to learn domain-specific name–concept mappings (e.g., that `ICS` is the Injury Causation Summary table and `SOE` is its Source-of-Energy column). We test $r \in \{8, 16, 32\}$ to find the sweet spot between expressivity and overfitting risk on our 301-example training set.

Learning rate (A1 vs. C1 vs. C2). A higher learning rate (2×10^{-4}, C1) converges faster but risks overwriting the model's pre-trained JSON-generation ability (catastrophic forgetting). A lower rate (5×10^{-5}, C2) updates the weights more gently. The default 10^{-4} is the standard LoRA learning rate; we bracket it to check whether it is optimal.

Target modules (A1 vs. D1). A1 applies LoRA to all seven projection types including the feed-forward layers (`gate_proj`, `up_proj`, `down_proj`). D1 restricts adaptation to the four attention projections only. Training fewer parameters reduces the risk of overfitting on a small dataset and was the default in our exploratory phase (which produced the 0.3983 baseline); we include it at the higher rank of 16 to check whether it remains competitive.

Best-combination config (E1). E1 combines the PK/FK format (motivated by joint-table reasoning) with rank 16 (motivated by capacity) and 150 training steps. All other configurations use 120 steps. The step budget is deliberately conservative: empirical exploratory runs showed that training beyond 150 steps on this 301-example dataset consistently *degrades* validation performance, consistent with overfitting — the model memorizes training examples and partially overwrites its

pre-trained JSON-generation capability. E1 uses 150 steps (the empirically safe upper bound) as the "more training" variant, while A1–D1 use 120 to stay well within the overfitting threshold.

3.4 Multi-Config API Usage

All nine configurations are passed as a `List` to `RFGridSearch(configs=config_set, trainer_type="SFT")` and launched with a single call to `experiment.run_fit()`. RapidFire AI serializes each configuration, spawns a worker process, and saves the final LoRA checkpoint to `runs/<run_id>/checkpoints/final_checkpoint/`. Training metrics (loss curves, evaluation loss at each checkpoint) are logged to `mlflow/` and are included in the submission's `logs/` directory.

After training, each adapter is evaluated with `main.py` using the `--format` flag matching its training format and scored with `eval.py` to produce per-question precision/recall/F1 at the table and column level.

4 Results and Discussion

[Results to be added after training and evaluation complete.]

5 Conclusion and Future Work

[To be added.]

References

[1] Qwen Team, Alibaba Cloud. *Qwen2.5 Technical Report*, 2024. https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

[2] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.

[3] SNAILS: Schema-Linking and Natural Language to SQL benchmark. Course materials, CSE/DSC 234, UC San Diego, Spring 2026.

[4] *json-repair*: A Python library for repairing broken JSON strings. https://github.com/mangiucugna/json_repair